

CSE437: Data Science Coding with Real world data

Data - is set of value. Becomes information when it has a meaning.
 Science - applying scientific methods to process data.
 → to gain insights into data thro computation, stats and visualization.

Phases of a data Science project:



Decision Making through Random forest: ^{Bootstrapping} random selection of data to make tree and repeat instance.

Classification - grouping discrete values. (for contⁿ we use regression)

Random forest is a nonlinear ML algo.

Bagging method: to make forest from many trees.

GINI impurity: of a node (similar to entropy^{in DT})

eg. $\begin{matrix} ++ \\ ++ \end{matrix}$ → a set with all 'yes' → Pure. $I_n = 0$
 $\begin{matrix} ++ \\ +- \end{matrix}$ → Lets say we want to pick '-',
 $P(\text{picking } + \text{ mistakenly}) = \frac{3}{5} \times \frac{2}{5}$
 Now we want '+'
 $P(\text{picking } - \text{ mistakenly}) = \frac{2}{5} \times \frac{3}{5}$

using formula $Gini \text{ impurity} = + = \frac{12}{25}$

GINI impurity of a node is the probability that a randomly chosen sample in a node would be incorrectly labelled.

$$I_G(p) = \sum_{i=1}^n P_i \sum_{k \neq i} P_k = 1 - \sum_{i=1}^n P_i^2$$

no. of classes, probab of picking item labelled i

$$I_G(D) = 1 - \sum_{i=1}^2 P_i^2 = 1 - (P_1^2 + P_2^2) = 1 - \left\{ \left(\frac{3}{5} \right)^2 + \left(\frac{2}{5} \right)^2 \right\} = \frac{12}{25}$$

ex: $\begin{matrix} ++ \\ +- \end{matrix}$ → $gini(D) = 0.32$ $Gini(D) = 1 - \sum_{i=1}^2 P_i^2$

If we split D to D₁ and D₂, $gini_A(D) = \frac{|D_1|}{|D|} gini(D_1) + \frac{|D_2|}{|D|} gini(D_2)$

$$gini_A(D) = \frac{1}{6} \times 0 + \frac{5}{6} \times 0.32 \quad gini_B(D) = \frac{4}{6} \times 0 + \frac{2}{6} \times 0$$

ex:

x ₁	x ₂	class
1	1	0
2	1	1
1	2	1
2	2	0
2	3	1
3	3	1

 $Gini(D) = (\text{same as above}) = 1 - \left(\left(\frac{1}{2} \right)^2 + \left(\frac{1}{2} \right)^2 \right)$ but the data is contⁿ. (In 4th page)

feature with lowest impurity

ex. (the dataset from CSF422)

Implement Random forest on the given dataset

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Day1	Sunny	Hot	High	Weak	No
Day2	Sunny	Hot	High	Strong	No
Day3	Overcast	Hot	High	Weak	Yes
Day4	Rain	Mild	High	Weak	Yes
Day5	Rain	Cool	Normal	Weak	Yes
Day6	Rain	Cool	Normal	Strong	No
Day7	Overcast	Cool	Normal	Strong	Yes
Day8	Sunny	Mild	High	Weak	No
Day9	Sunny	Cool	Normal	Weak	Yes
Day10	Rain	Mild	Normal	Weak	Yes
Day11	Sunny	Mild	Normal	Strong	Yes
Day12	Overcast	Mild	High	Strong	Yes
Day13	Overcast	Hot	Normal	Weak	Yes
Day14	Rain	Mild	High	Strong	No

$$GINI(D) = 1 - \left\{ \left(\frac{2}{14} \right)^2 + \left(\frac{5}{14} \right)^2 \right\} = 0.456$$

Randomly pick some rows and some columns. (Day 10-14, 2) Bootstrap Data 1

$$GINI(D) = 1 - \left\{ \left(\frac{4}{6} \right)^2 + \left(\frac{2}{6} \right)^2 \right\} = 0.444$$

$$GINI(D_1) = 1 - \left\{ \left(\frac{3}{4} \right)^2 + \left(\frac{1}{4} \right)^2 \right\} = 0.375$$

$$GINI(D_2) = 1 - \left\{ \left(\frac{1}{2} \right)^2 + \left(\frac{1}{2} \right)^2 \right\} = 0.5$$

$$\therefore GINI_{Temp}(D) = \frac{4}{6} \times 0.375 + \frac{2}{6} \times 0.5 = 0.417$$

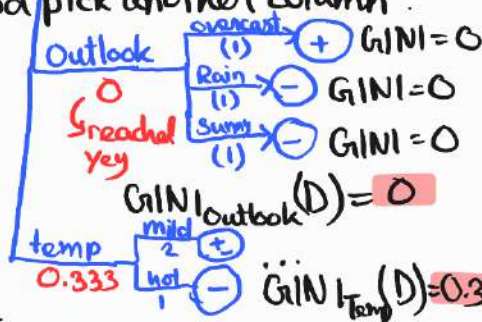


now from our bootstrap data 1 pick the rows where Humidity is high and pick another column.

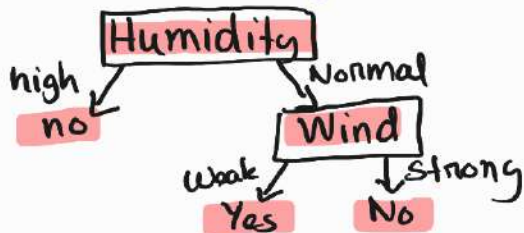
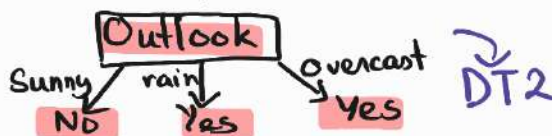
Day	Outlook	Temperature	Play Tennis
Day12	Overcast	Mild	Yes
Day14	Rain	Mild	No
Day2	Sunny	Hot	No



Decision Tree no. 1



Now take another set of rows for Bootstrap data 2 to make DT2. ex. Day 1-5, 2 and we get, and repeat..



Making Prediction using this forest of DT-1, 2, 3:

Outlook	Temp	Hum	Wind	Predict
Sunny	Hot	Normal	Strong	?

Bagging:

From DT-1 we get: Yes=1, No=0

From DT-2 we get: Yes=0+1, No=1+0

From DT-3 we get: Yes=0+1, No=1+1

high value for No
∴ prediction = No

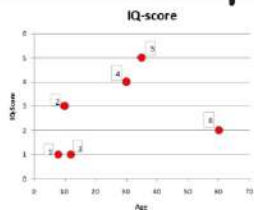
Classification: when output is discrete
Regression: when output is continuous

numerical
Need RT

So, we use DT

→ Continuous
So, its regression
(not classification)

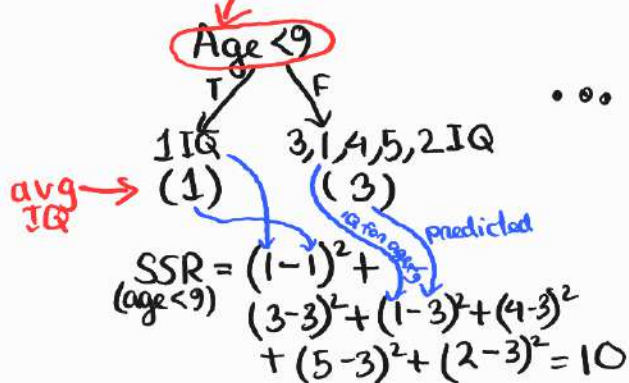
2. Get avg of 2 consecutive ages.



(non linear regression)

Age	avg
8	9
10	11
12	21
30	32.5
35	47.5
60	

- ↳ sum of sq residues



∴ to get,

	<u>Avg for left side (T)</u>	<u>Avg for right side (F)</u>	<u>SSR</u>
age < 9	1	3	10
age < 11	2	3	12
age < 21	1.6667	3.6667	7.3333
age < 32.5	2.25	3.5	11.25
age < 47.5	2.8	2	12.8

13, 14, 15

T

F

- $+ (5-3)^2 + (2-3)^2 = 10$
 4. Get the min SSR. age < 21
 min SSR = 7.33
 5. Repeat for the other independent var.

Observations	Age	Place of living	IQ-score
1	8	Village	1
2	10	City	3
3	12	Village	1
4	10	Village	4
5	35	City	5
6	60	City	2

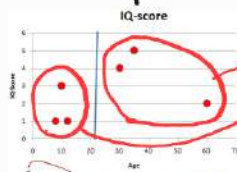
category converted
to no. on lvl

prediction: (2)

We can use village as the splitter.
 # of splitters = $2^{n-1} - 1$
 (Place of living = village) (diff # of values)

SSR = 10.667
(min SSR)

6. Compare the min SSR of all the independent vars. Root of the tree will be the one with smallest SSR. and split the dataset following that
- ```
graph TD; A[Age < 21] -- T --> B[1.67]; A -- F --> C[3.67];
```



We get two datasets from here. Repeat on them both.

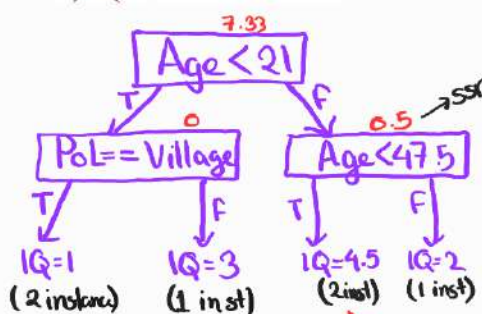
both. e.g. age 8

| <u>IQ</u> | <u>avg for T</u> | <u>avg for F</u> | <u>SSR</u> |
|-----------|------------------|------------------|------------|
| 1         | 1                | 2                | 2          |
| 3         | 2                | 1                | 2          |

$$\min SSR = 2 \quad (\text{can take any one})$$

And for village? splitter we get  $SSR=0$ . Pick that for left node. Repeat on the other dataset piece for right not.

(we can stop extending the node if  $SSR < 2$ )

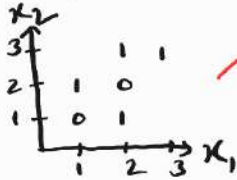


→ We stopped here coz, threshold was 2 instances.



# Random Forest for cont<sup>n</sup> data:

ex:



| $x_1$ | $x_2$ | class |
|-------|-------|-------|
| 1     | 1     | 0     |
| 2     | 1     | 1     |
| 2     | 2     | 1     |
| 2     | 3     | 1     |
| 3     | 3     | 1     |

6 instance, 2 classes

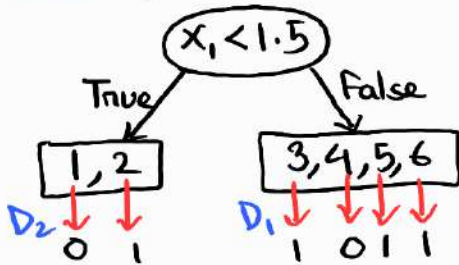
| idx | $x_1$ | class |
|-----|-------|-------|
| 1   | 1     | 0     |
| 2   | 1     | 1     |
| 3   | 2     | 1     |
| 4   | 2     | 1     |
| 5   | 2     | 1     |
| 6   | 3     | 1     |

The instance can also take float values.  $\therefore$  Continuous

1. arrange them  
e.g. for  $x_1$ , ascending order.

2. get avg of 2 consecutive numbers.

5. Repeat



$$GINI(D_1) = \dots = 0.375$$

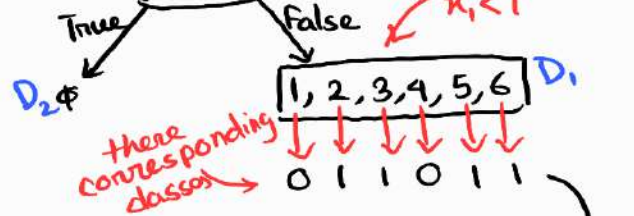
$$GINI(D_2) = 0.5 \rightarrow (\text{as same no. of 0 and 1})$$

$$GINI_{x_1 < 1.5}(D) = 0.417$$

$$\text{Similarly, } GINI_{x_1 < 2} = 0.417$$

$$GINI_{x_1 < 2.5} = 0.4$$

3.  $x_1 < 1$



4. Calc. GINI of those sets.

$$GINI(D) = 1 - \left\{ \left( \frac{4}{6} \right)^2 + \left( \frac{2}{6} \right)^2 \right\} = 0.444$$

And then  $D_2$  (but  $D_2$  is 0 here)

$$GINI_{x_1 < 1}(D) = \frac{6}{6} \times 0.444 + 0 = 0.444$$

6. Sort again for  $x_2$  and get their GINI

| idx | $x_2$ | class |
|-----|-------|-------|
| 1   | 1     | 0     |
| 2   | 1     | 1     |
| 3   | 2     | 1     |
| 4   | 2     | 1     |
| 5   | 3     | 1     |
| 6   | 3     | 1     |

$$GINI_{x_2 < 1}(D) = 0.444$$

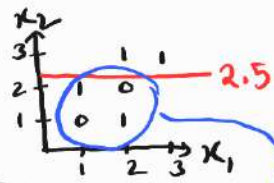
$$GINI_{x_2 < 1.5}(D) = 0.417$$

$$GINI_{x_2 < 2}(D) = 0.417$$

$$GINI_{x_2 < 2.5}(D) = 0.333$$

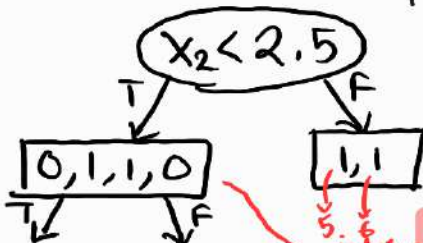
$$GINI_{x_2 < 3}(D) = 0.333$$

7. Compare both to get root and split



$\therefore x_2 < 2.5$  should be root

we consider this part only now.



| idx | $x_1$ | $x_2$ | cls |
|-----|-------|-------|-----|
| 1   | 1     | 1     | 0   |
| 2   | 1     | 2     | 1   |
| 3   | 2     | 2     | 1   |
| 4   | 2     | 2     | 1   |

8. Repeat on this shortened data to get the next nodes.

$$GINI_{x_1 < 1}(D) = 0.5$$

$$GINI_{x_1 < 1.5}(D) = 0.5$$

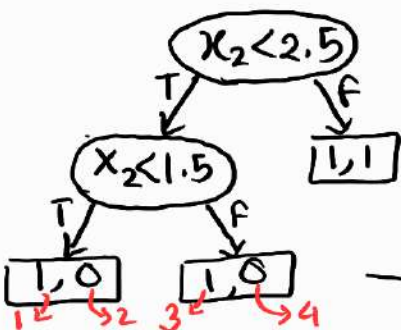
$$GINI_{x_1 < 2}(D) = 0.5$$

$$GINI_{x_2 < 1}(D) = 0.5$$

$$GINI_{x_2 < 1.5}(D) = 0.5$$

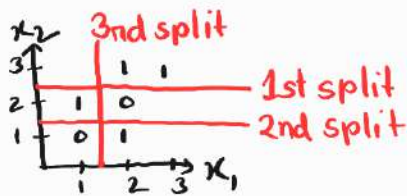
$$GINI_{x_2 < 2}(D) = 0.5$$

Same values. So, pick any. e.g.  $x_2 < 1.5$

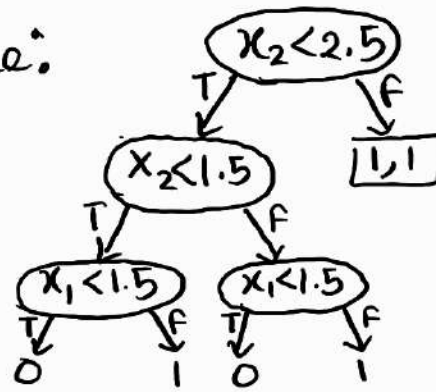


9. Split again in 2 tables. Repeat for both.





Final tree:



Boosting algo for binary classification for both classification and regression

## Ada Boost (Adaptive Boosting)

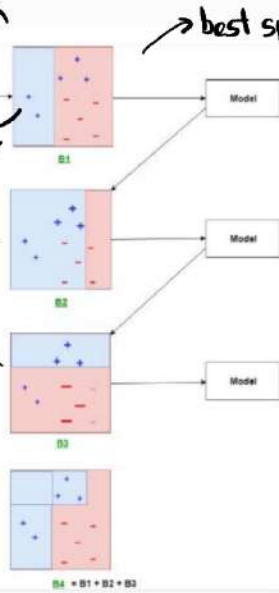
ensemble modelling technique to build a strong classifier from a no. of weak classifiers  
Other boosting algo: Gradient tree boosting, XGBoost

weak learner → only consider 1 classifier.

this will classify the 2 correctly (and wrongly for the 3 + one neg side)

combine the 3 weak classifier

strong classifier



1. Assign equal weight (attention) to each data point. ( $1/n$ )
2. Make the weak classification and identify the wrongly classified data and increase their weights
3. Repeat until we get required results.

### RF vs AdaBoost:

In AdaBoost, the trees have just 1 node and two leaves. Unlike RF, AdaBoost use weights in bagging. (RF trees have eq weight)  
RF makes trees randomly, but in adaBoost next tree depends on previous tree.

ex. Implement Random forest on the given dataset

| Day   | Outlook  | Temperature | Humidity | Wind   | Play Tennis |
|-------|----------|-------------|----------|--------|-------------|
| Day1  | Sunny    | Hot         | High     | Weak   | No          |
| Day2  | Sunny    | Hot         | High     | Strong | No          |
| Day3  | Overcast | Hot         | High     | Weak   | Yes         |
| Day4  | Rain     | Mild        | High     | Weak   | Yes         |
| Day5  | Rain     | Cool        | Normal   | Weak   | Yes         |
| Day6  | Rain     | Cool        | Normal   | Strong | No          |
| Day7  | Overcast | Cool        | Normal   | Strong | Yes         |
| Day8  | Sunny    | Mild        | High     | Weak   | No          |
| Day9  | Sunny    | Cool        | Normal   | Weak   | Yes         |
| Day10 | Rain     | Mild        | Normal   | Weak   | Yes         |
| Day11 | Sunny    | Mild        | Normal   | Strong | Yes         |
| Day12 | Overcast | Mild        | High     | Strong | Yes         |
| Day13 | Overcast | Hot         | Normal   | Weak   | Yes         |
| Day14 | Rain     | Mild        | High     | Strong | No          |

Data,  $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$

•  $N=14$ ,  $w_i = 1/N$  (initial weight)

• For  $m=1:M$

Fit classifier  $G_m(x) \in \{-1, 1\}$  using weights,  $w_i$ .

$$err_m = \frac{\sum_{i=1}^N w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^N w_i}$$

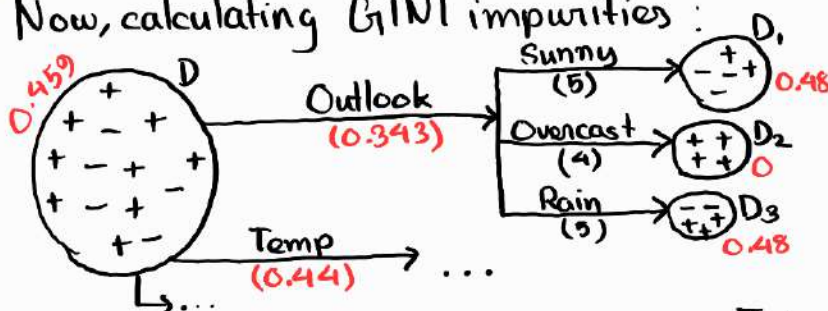
No → Yes  
1 if wrongly classifies.  
0 otherwise

$$\alpha_m = \log\left(\frac{1 - err_m}{err_m}\right)$$

err side

$$\text{Set } w_i \leftarrow w_i \exp[\alpha_m I(y_i \neq G_m(x_i))]$$

Now, calculating GINI impurities:



$$GINI_{outlook}(D) = \dots = 0.343$$

$$GINI_{temp}(D) = 0.44$$

$$GINI_{humid}(D) = 0.367$$

$$GINI_{wind}(D) = 0.429$$

Take the smallest one.



Weak learner 1

$G_1(x)$



prediction by  $G_1(x)$

$G_1(x)$

actual output  $y$

|      |     |     |   |
|------|-----|-----|---|
| Day1 | No  | No  | ✓ |
| Day2 | No  | No  | ✓ |
| Day3 | Yes | Yes | ✓ |
| Day4 | Yes | Yes | ✓ |
| Day5 | Yes | Yes | ✓ |
| Day6 | Yes | No  | ✗ |

like this we get 4 errors in day 6, 9, 11, 14

amount of say (how well it classified)

$$err_1 = \frac{4 \times \frac{1}{14}}{1} = 0.286$$

$$\alpha_1 = \frac{1}{2} \ln \left( \frac{1 - 0.286}{0.286} \right) = 0.45$$

$$w_1 = w_1 \cdot e^{\alpha_1} = \frac{1}{14} e^{0.45} = 0.11$$

initial weights were all  $\frac{1}{14}$

$$w_1 \text{ for corrected guess} = w_1 \cdot e^{-0.45} = 0.046$$

for the other days

Now normalize this column of new weights,  $Nor = \frac{\text{weight}}{\text{total weight}}$

Now get the cumulative weight of these normalised weights.

Then generate random numbers 0-1 for each data point.

Using these two columns, we will create our new dataset:

| Day | Sample Weight | Sample Weight | Normalized Sample Weight | Cumulative Normalized Sample Weight | Generated Random Number |
|-----|---------------|---------------|--------------------------|-------------------------------------|-------------------------|
| 1   | 1/14          | 0.046         | 0.051                    | 0.051                               | 0.040                   |
| 2   | 1/14          | 0.046         | 0.051                    | 0.102                               | 0.100                   |
| 3   | 1/14          | 0.046         | 0.051                    | 0.153                               | 0.151                   |
| 4   | 1/14          | 0.046         | 0.051                    | 0.204                               | 0.200                   |
| 5   | 1/14          | 0.046         | 0.051                    | 0.255                               | 0.250                   |
| 6   | 1/14          | 0.11          | 0.122                    | 0.377                               | 0.267                   |
| 7   | 1/14          | 0.046         | 0.051                    | 0.428                               | 0.500                   |
| 8   | 1/14          | 0.046         | 0.051                    | 0.479                               | 0.700                   |
| 9   | 1/14          | 0.11          | 0.122                    | 0.601                               | 0.990                   |
| 10  | 1/14          | 0.046         | 0.051                    | 0.652                               | 0.370                   |
| 11  | 1/14          | 0.11          | 0.122                    | 0.774                               | 0.600                   |
| 12  | 1/14          | 0.046         | 0.051                    | 0.825                               | 0.682                   |
| 13  | 1/14          | 0.046         | 0.051                    | 0.876                               | 0.886                   |
| 14  | 1/14          | 0.11          | 0.122                    | 0.998                               | 0.980                   |

for each random no., find in which interval it belongs to, and take the upper bound datapoint.

in interval 0 to 0.051 (Day1). So, we take Day1

in 0.051 to 0.1 (Day2). So, pick Day2  
... (continue in this seq)

in 0.876 to 0.998. So pick Day14  
...

This way, we get:

Day 1  
2  
3  
4  
5  
6  
9  
11  
14  
6  
9  
11  
14  
14

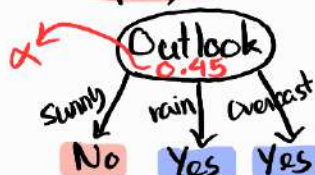
So, now we have a new dataset (some are missing e.g. 7, 8, 10, 12, 13 and some are repeated e.g. 6, 9, 11, 14 which were the misclassified ones).

Repeat the model on this to get,  $G_2(x)$ ...

$G_1(x)$

$G_2(x)$

$G_3(x)$



In dataset 1

In dataset 2

this happened coz its weight was bigger than correctly classified ones so, the interval was bigger and it appeared as the upper bound for more than 1 time.

Day 30: Out=Overcast  
Temp=high  
Humidity=Norm  
Wind=Strong

Pass this through all 3 weak learners:

$G_1(x) \rightarrow \text{Yes}$  (0.45),  $G_2(x) \rightarrow \text{Yes}$  (0.23),  $G_3(x) \rightarrow \text{No}$  (0.56)  
Yes = 0.45 + 0.23 = 0.68  
No = 0.56  
∴ Prediction: Yes (take higher)



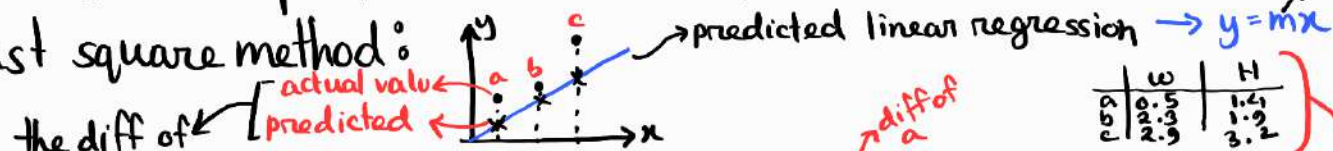
# Gradient Descent

Logistic Regression to predict categorical dependent var using indep. vars.  
 $\frac{dy}{dx}$  → rate of change of y w.r.t. x. We can use it to know if the func is increasing/decreasing



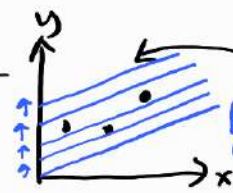
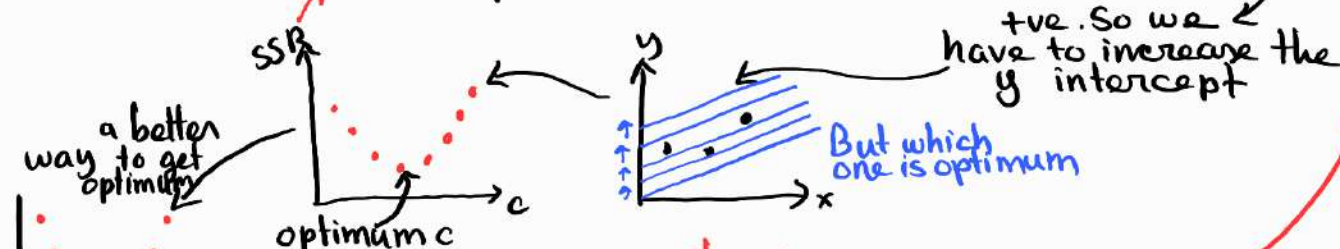
Gradient descent: an algo that uses gradient to descent to the lowest point of an error function.

Least square method:



|   | w   | H   |
|---|-----|-----|
| a | 0.5 | 1.4 |
| b | 2.3 | 1.9 |
| c | 2.9 | 3.2 |

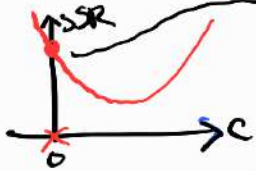
Sum of squared residues =  $1.1^2 + 0.4^2 + 1.3^2 = 3.1$



+ve. So we have to increase the y intercept

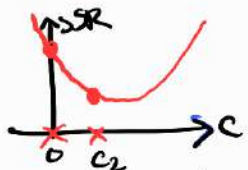
$$SSR = \{1.4 - (c + 0.64 \times 0.5)\}^2 + \{1.9 - (c + 0.64 \times 2.3)\}^2 + \{3.2 - (c + 0.64 \times 2.9)\}^2$$

Differentiate this w.r.t c. Get  $\frac{d}{dc} SSR$  for c=0



Gradient at c=0, -5.7

- Step size =  $-5.7 \times 0.1 = -0.57$
- New intercept,  $c_2 = c_1 - \text{step size} = 0 - (-0.57) \Rightarrow c_2 = 0.57$



Repeat to get smaller and smaller steps to get optimum c value (int we take 1000 steps)

But see also need to change the slope:

$$1. SSR = \{1.4 - (c + m \times 0.5)\}^2 + \dots + \{3.2 - (c + m \times 2.9)\}^2$$

And get  $\frac{dSSR}{dc}$  and  $\frac{dSSR}{dm}$

|                | w   | H   |
|----------------|-----|-----|
| P <sub>1</sub> | 0.5 | 1.4 |
| P <sub>2</sub> | 2.3 | 1.9 |
| P <sub>3</sub> | 2.9 | 3.2 |



$$h = mw + c$$

- 2. Plug in c=0 in  $\frac{dSSR}{dc}$  and m=1 in  $\frac{dSSR}{dm}$ . We get  $\frac{dSSR}{dc} = -1.6$ ,  $\frac{dSSR}{dm} = -0.8$  and then,  $\text{Step size}_c = -1.6 \times \text{learning rate}$ ,  $\text{Step size}_m = -0.8 \times \text{learning rate}$

- 3. New intercept =  $c_0 - (-0.016) = 0 - (-0.016) = 0.016$ , New slope =  $m_0 - (-0.008) = 1 - 0.008 = 0.992$
- 4. Repeat until step size very small or max no. of steps reached



What if we also consider the weight, our SSR will be like,

|       | BMI | W   | H   |
|-------|-----|-----|-----|
| $p_1$ | 1.6 | 0.5 | 1.4 |
| $p_2$ | 1.2 | 2.3 | 1.9 |
| $p_3$ | 0.8 | 2.9 | 3.2 |

→ output

$$SSR = \{1.4 - (c + s_1 \times 0.5 + s_2 \times 1.6)\}^2 + \dots + \{3.2 - (c + s_1 \times 2.9 + s_2 \times 0.8)\}^2$$

Now we have 3 unknowns. So, we'll need  $\frac{\partial SSR}{\partial c}$ ,  $\frac{\partial SSR}{\partial s_1}$ ,  $\frac{\partial SSR}{\partial s_2}$

Start with  $c=0$ ,  $s_1=s_2=1$  and continue.

• SSR is a loss function but, there are others than can be used too.

## Gradient Descent

Regression Losses  
Mean Square Error/Quadratic Loss/L2 Loss

Mathematical formulation :-

$$MSE = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}$$

Mean Squared Error

Mean Absolute Error/L1 Loss

Mathematical formulation :-

$$MAE = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{n}$$

Mean absolute error

$$MBE = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)}{n}$$

Mean bias error

Mathematical formulation :-

$$SVM Loss = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

SVM Loss or Hinge Loss

Mathematical formulation :-

$$CrossEntropyLoss = -(y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

Cross entropy loss

## Gradient Descent steps:

1. Get derivative of Loss func w.r.t to each unknown parameter.
2. Pick random val for them
3. Step size = derivative  $\times$  learning rate
4. New param = Old param - Step size

• Stochastic Gradient descent: inl there are way more points to deal with than 3. In this method, <sup>few</sup> random points are picked <sup>at every step</sup>.

• Least square method: to get optimum values. Doesn't consider loss function.

| x         | y         | $x - \bar{x}$ | $y - \bar{y}$ | $(x - \bar{x})^2$ | $(x - \bar{x})(y - \bar{y})$ |
|-----------|-----------|---------------|---------------|-------------------|------------------------------|
| $\bar{x}$ | $\bar{y}$ |               |               |                   |                              |

independent var

dependent var

calculate these

(not used anymore)

$$y = mx + c \rightarrow m = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sum (x - \bar{x})^2}, c = \bar{y} - m\bar{x}$$

• Nearest-Neighbour Classifier: Instance based classifier, where the training data are saved and used for predictions e.g. K-Nearest (predicts when it exactly matches a train data), Nearest neighbour (closest points to test case)



unknown data.

$k=3$

By majority voting, the prediction will be +.

(If +, - are both 50%, pick any)

dist from unknown to all points are calculated for this

and the identify k nearest neighbours to determine unknown point's label

1-nearest neigh

2-nearest neigh

Another way of getting 1 Nearest neigh: Voronoi Diagram  
Whichever region the unknown pt will be at, that will be its prediction.



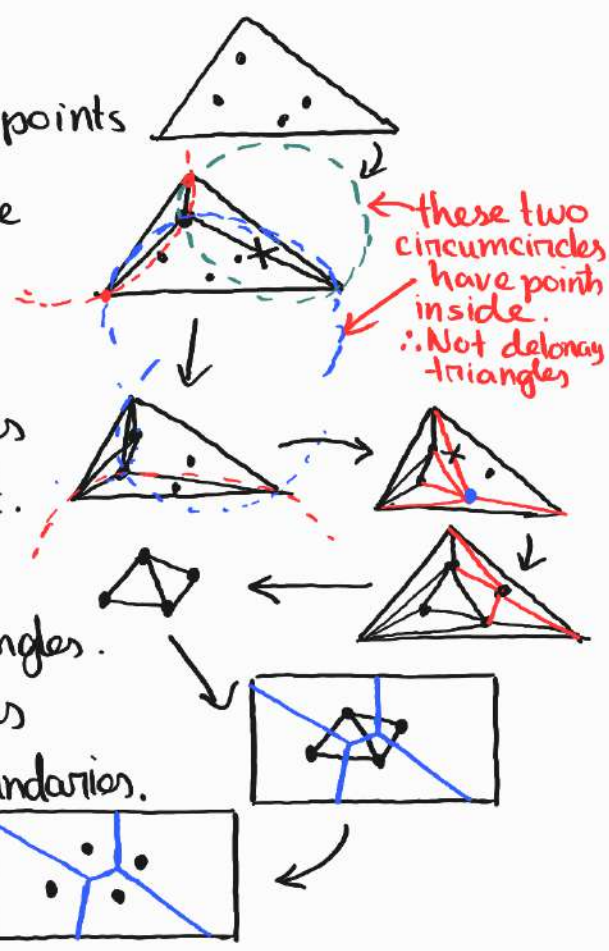
# Voronoi diag: A planar partition. Region have 1 point.

We need Delounay triangulation for it.  
 where none of the points are inside the circumcircle of the triangle



### Bower-Watson Algorithm:

1. first we draw the supertriangle around all points
2. Select a point randomly and join the edges and then check if they are delounay triangles
3. Remove the non delounay triangles and select another point and repeat.
4. At the end, remove the supertriangle
5. Now we have all the delounay triangles.  
 For each edge, draw their bisectors and extend them upto the set boundary.
6. Remove the delounay triangles.



Voronoi Diag

### 1-Nearest Neigh:

Whichever region the unknown pt will be at, that will be its prediction.

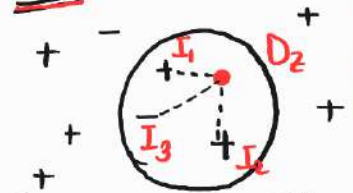
Several distance functions,  $d(p, q)$ :

- Eucadian  $\rightarrow \sqrt{\sum (p_i - q_i)^2}$
- Manhattan  $\rightarrow \sum |p_i - q_i|$
- q-norm  $\rightarrow (\sum |p_i - q_i|^q)^{1/q}$
- Cosine diff  $\rightarrow \frac{p \cdot q}{\|p\|_2 \|q\|_2}$

other data points (red arrow)  
 unknown point (red arrow)  
 e.g. 3-norm (blue arrow)  
 higher value  $\rightarrow$  more similarity (red arrow)

### Majority Voting:

ex:



for  $v = '+'$ ,  $\sum_{(x_i, y_i) \in D_z} I(+ = y_i) = 1 + 1 + 0 = 2$   
 for  $v = '-'$ ,  $\sum_{(x_i, y_i) \in D_z} I(- = y_i) = 0 + 0 + 1 = 1$

$\arg \max_v \rightarrow v = +$  as  $2 > 1$ ,  $\therefore$  Unknown point  $\rightarrow +$

$$y' = \arg \max_v \sum_{(x_i, y_i) \in D_z} I(v = y_i)$$

calculated for each v (labels of y) and the v with max is picked

inputs (blue arrow)  
 outputs (blue arrow)

if  $v = y_i$  then  $I(v = y_i) = 1$ , 0 otherwise.  
 set of K-closest to z

Weighted voting:  $w = 1/d^2$   
 $y' = \arg \max_v \sum_{(x_i, y_i) \in D_z} w_i \times I(v = y_i)$   
 distance from  $I_i$  to unknown pt z.



## KNN classifier algorithm:

for each test ex.,  $z = (x', y')$  do,  
 Compute  $d(x', x)$  for every point  $(x, y)$  in  $D$   
 Select  $D_z \subseteq D$   
 $y' = \arg \max_v \sum_{(x_i, y_i) \in D_z} I(v = y_i)$   
 end for

$\rightarrow$  in  $D$ , dataset

$\rightarrow$  set of  $k$  closest points (training data) to  $z$

$\rightarrow$  on the weight voting (if mentioned)  
 if not mentioned, and there's a tie, used weighted voting



- If  $k$  value is too small, it might be sensitive to noise points. If too large, it might also include other classes.    
 → less sensitive to noise
- Scaling issues: all the attributes may need to be scaled so that one attribute's dist measure don't dominate the others.   
 (normalise) → by s.d.   
 e.g. height 1.5-1.8m, weight 60-100kg
- Problems with Euclidean measure: (others too when too many irrelevant attributes) Might give undesirable results at higher dimensions.   
 e.x.  $P = 111111111110$  vs  $P = 1000000000000$    
 $Q = 011111111111$  vs  $Q = 000000000000$    
 → Both  $d = 1.4142$  even tho very diff
- Sol<sup>n</sup>: remove irrelevant attri in preprocessing. Give them diff  $w_i$ . Increases  $k$ .
- $k$ -NN classifiers are lazy learners: doesn't build models explicitly,   
 It's unlike eager learners e.g. Decision tree induction and rule-based systems. Relatively expensive for <sup>classifying</sup> unknown points.   
 If dataset too big, we can bootstrap. More attr → Accuracy degrades
- For discrete data → hamming dist,  $d(p, q) = \text{no. of features on which } p, q \text{ differ}$    
 e.g. similarity metric: no. of matching attri,  $k=2$

|   | Food<br>(3) | Chat<br>(2) | Fast<br>(2) | Price<br>(3) | Bar<br>(2) | BigTip |
|---|-------------|-------------|-------------|--------------|------------|--------|
| 1 | great       | yes         | yes         | normal       | no         | yes    |
| 2 | great       | no          | yes         | normal       | no         | yes    |
| 3 | mediocre    | yes         | no          | high         | no         | no     |
| 4 | great       | yes         | yes         | normal       | yes        | yes    |

- (great, no, no, normal, no)

1 mismatch  $\rightarrow$  'yes'  
2 mismatch  $\rightarrow$  'yes' }  $\therefore$  Prediction = yes



Before Mid we did supervised learning.

**Clustering**: unsupervised learning. → no labels

Find labels and no. of clusters from data



Divide the data pts into groups based on similarities!

High intra group similarity and low inter group similarity

Finding natural groupings among objects.

e.g. a bunch of people pic — cluster based on male or female

"

"

" smoker or not

Similarity: of features, resemblance

e.g. Distance measures:  $D(O_1, O_2) \rightarrow$  Dissimilarities bet<sup>m</sup>  $O_1$  &  $O_2$ .

Properties:

otherwise A could look like B while B doesn't look like A

1. Symmetry,  $D(A, B) = D(B, A)$

2.  $D(A, A) = 0$

3. Separation,  $D(A, B) = 0$  iff  $A = B$

triangle inequality → 4.  $D(A, B) \leq D(A, C) + D(B, C)$

Euclidean, (a.k.a 2-norm dist)

$$d(x, y) = \sqrt{\sum |x_i - y_i|^2}$$

→ Manhattan, (a.k.a taxicab norm or 1-norm)

$$d(x, y) = \sum |x_i - y_i|$$

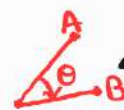
→ Maximum norm,  $d(x, y) = \max |x_i - y_i|$

connects data for different scales & correlation in vars — Mahalanobis

a.k.a. edit dist. Min no. of substi req. to change 1 member to another — Hamming

→ Inner prod space — angle bet<sup>m</sup> two vectors (for high dim data)

→ Cosine similarity,



$$\cos(\theta) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

Clustering 2 types: Hierarchical, Partitional algos



creates a hierarchical decomposition

of the sets of objs using some criterion. Finds successive clusters using prev clusters.

• Agglomerative ("bottom up") — begins with each element as separate cluster and merge them successively

• Division ("top-down") — begins with whole set and divides into smaller clusters

→ construct many partitions and then evaluate them by some criterion.



Finds all clusters at once.

• K-means and derivatives  
• Fuzzy c-means clustering  
• QT clustering algo

• **K-means**: clusters  $n$  objs based on attributes into  $k$  disjoint parts in such a way, that it minimizes sum of squares (sum of sq. of dist bet<sup>m</sup> data & center)

→ assume they form a vector space

Find centers of the clusters.

(Similar to Expectation-maximization algo)

$$J = \sum_{j=1}^K \sum_{n \in S_j} |x_n - M_j|^2$$

→ vector of  $n^{\text{th}}$  data  
→ geometric centroid in  $S_j$   
→ for each cluster



e.g. 1, 2, 5, 9, 7, 3  $\rightarrow S_1 = \{1, 2, 5\}$   $\rightarrow S_2 = \{9, 7, 3\}$   $\rightarrow$  two clusters of the dataset

$$J_1 = |1 - 2.67|^2 + |2 - 2.67|^2 + |5 - 2.67|^2 = 18.67$$

$$J_2 = |9 - 6.33|^2 + |7 - 6.33|^2 + |3 - 6.33|^2 = 27.33$$

46

Make another clusters from the dataset and find their  $J$  to get the cluster with min  $J$ .  $\rightarrow$  K-means does this efficiently.

- K-means algo:
  - decide k value
  - Initialize the centroids of the clusters (Randomly from the dataset if needed)
  - Assign each objects to the nearest cluster center
  - Re-estimate cluster centers by calculating dist from each center
  - Repeat from step 3 until non of the objs change clusters .exit.

ex. Dataset:

| Individual | Variable 1 | Variable 2 |
|------------|------------|------------|
| 1          | 1.0        | 1.0        |
| 2          | 1.5        | 2.0        |
| 3          | 3.0        | 4.0        |
| 4          | 5.0        | 7.0        |
| 5          | 3.5        | 5.0        |
| 6          | 4.5        | 5.0        |
| 7          | 3.5        | 4.5        |

1. Let  $k=2$  and randomly pick 2 as centers  $m_1, m_2$   
 $(1, 1)$   $(5, 7)$

2. Distance of each point from both centers.

$$d(2, m) = \sqrt{|1 - 1.5|^2 + |1 - 2|^2} = 1.12$$

|   | $m_1$<br>(1, 1) | $m_2$<br>(5, 7) |
|---|-----------------|-----------------|
| 1 | 0               | 7.21            |
| 2 | 1.12            | 6.10            |
| 3 | 3.61            | 3.61            |
| 4 | 7.21            | 0               |
| 5 | 4.72            | 2.5             |
| 6 | 5.81            | 2.05            |
| 7 | 4.30            | 2.92            |

$\rightarrow$  pick the closest center

3. So, we get 2 clusters (1st Iteration)

Cluster 1,  $S_1 = \{1, 2, 3\}$   $\rightarrow$  new center,  $m_1 =$

Cluster 2,  $S_2 = \{4, 5, 6, 7\}$   $\rightarrow$  new center,  $m_2 = (4.12, 5.38)$

|     | Var1 | Var2 |
|-----|------|------|
| 1   | 1    | 1    |
| 2   | 1.5  | 2    |
| 3   | 3    | 4    |
| avg | 1.83 | 2.33 |

4. Recalculate dist from new centroids,  $= (1.83, 2.33)$

|   | $m_1$<br>(1.83, 2.33) | $m_2$<br>(4.12, 5.38) |
|---|-----------------------|-----------------------|
| 1 | 1.57                  | 5.38                  |
| 2 | 0.47                  | 4.28                  |
| 3 | 2.04                  | 1.78                  |

New clusters: (2nd Iteration)

$S_1 = \{1, 2\} \rightarrow m_1 = (1.25, 1.5)$

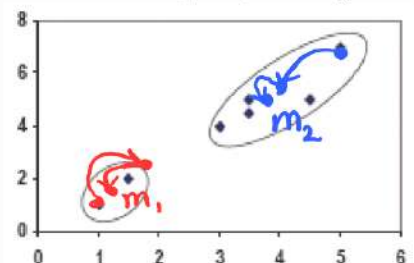
$S_2 = \{3, 4, 5, 6, 7\} \rightarrow m_2 = (3.9, 5.1)$

5. (3rd Iteration)

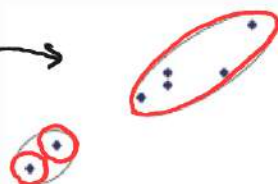
$S_1 = \{1, 2\}$

$S_2 = \{3, 4, 5, 6, 7\}$

$\rightarrow$  no change  $\therefore$  This is the final cluster.



• If we repeated with  $K=3 \rightarrow$

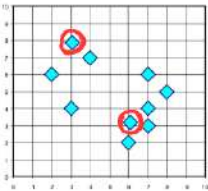




But K-Means is sensitive to outliers. Solution: K-Medoids which takes medoids instead of mean of the points in a cluster.

• K-Medoids : (PAM) Partitioning Around Medoids

$$C = \sum_i \sum_{p \in C_i} |p_i - c_i|$$



K=2

1. Randomly pick K=2 objects as initial medoids
2. Assign other objects to the nearest medoids. Calc cost.
3. Among the non-medoids, randomly select one. Calc cost. If quality improved, do the swap. Repeat.

ex:

|   | $x_1$ | $x_2$ | d from $C_1$ | d from $C_2$ |
|---|-------|-------|--------------|--------------|
| 0 | 7     | 6     | 6            | 2            |
| 1 | 2     | 6     | 3            | 7            |
| 2 | 3     | 8     | 4            | 8            |
| 3 |       |       |              |              |

$$\sum |x_i - c_i|$$

K=2  
Randomly picked:  $C_1 \rightarrow (3, 4)$   
two points  $C_2 \rightarrow (7, 4)$

$$\text{Dissimilarity: } d = |7-7| + |6-4| = 2$$

d is less for  $C_1$ . So, the object  $i=2$  belongs to  $C_1$  cluster.

So, the points 1, 2, 5 go to cluster  $C_1$  and 0, 3, 6, 7, 8 to  $C_2$ .

$$\text{Cost, } C = (3+4+4) + (3+1+1+2+2) = 20$$

Now keeping a point randomly: (7, 3) from  $C_2$

| $x_1$ | $x_2$ | d from $C_1$ | d from (7, 3) |
|-------|-------|--------------|---------------|
| 7     | 6     | 6            | 3             |
|       |       | ...          | ...           |

(Same as before)

New clusters:

$C_1 \rightarrow 1, 2, 5$

(7, 3)  $\rightarrow$  0, 3, 6, 7, 8

$$\text{New cost, } C = (3+4+4) + (2+2+1+3+3) = 22 \quad \text{new dissimilarities} \quad \text{cost increased (Bad)}$$

$$\text{Swap cost} = \text{Present cost} - \text{Prev Cost} = 22 - 20 = 2 > 0$$

So, we undo the swap

Final medoids  $\rightarrow C_1(3, 4), C_2(7, 4)$

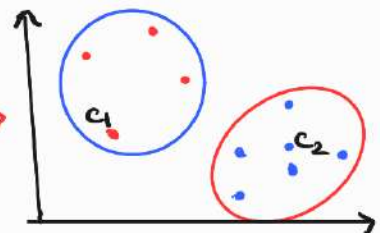
• Algo: Select K random points as medoids  
Associate each point to nearest medoids  
While cost decreases:

for each  $C_i$ , for each non medoid o:

Swap  $C_i$  and o

Recompute cost

If cost > prev cost: undo swap



Time complexity:  $O(k * (n-k)^2)$

K-Medoid : Advantages Disadvantages

• simple, easy to implement

• Fast, Converges in a fixed no. of steps.

• PAM less sensitive to outliers

• Not suitable for clustering non spherical groups of objs. coz it relies on minimizing dist bet<sup>m</sup> medoids. Uses compactness, not connectivity

• Give diff result on same data as medoid picked randomly



PAM works effectively for small data sets, but too much computational complexity for larger data sets.

Improve PAM efficiency  $\rightarrow$  CLARA : PAM on samples

CLARANS : Randomized re-sampling

Both K-means, K-medoids are simple and useful for undirected knowledge discovery. Used in pattern recognition, machine vision etc.

## Principal Components Analysis (PCA):

Useful for finding patterns in data of high dim. Highlight simil. & diff.

Mean,  $\bar{x} = \frac{\sum_{i=1}^n x_i}{n}$

Variance,  $s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n-1}$

s.d = s

$\hookrightarrow$  only operates on 1D data.

Covariance is how much two dimensions vary from the mean w.r.t each other

$$\text{cov}(x, y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{(n-1)}$$

If +ve, both dim  $\uparrow$  together. If -ve, one  $\uparrow$  other one  $\downarrow$ .  
If 0, the dims are independent.

Covariance Matrix for 3 vars  $x, y, z$  :  $C = \begin{matrix} & \begin{matrix} x & y & z \end{matrix} \\ \begin{matrix} x \\ y \\ z \end{matrix} & \begin{pmatrix} \text{cov}(x,x) & \text{cov}(x,y) & \text{cov}(x,z) \\ \text{cov}(y,x) & \text{cov}(y,y) & \text{cov}(y,z) \\ \text{cov}(z,x) & \text{cov}(z,y) & \text{cov}(z,z) \end{pmatrix} \end{matrix}$

Eigenvector of a sq matrix  $\rightarrow$  the vector's direction doesn't change under the associated linear transformation.  $Av = \lambda v$

Finding  $\lambda$  :  $\det(A - \lambda I) = 0$  (Shortcut for 2x2:  $\lambda^2 - \tau\lambda + \Delta = 0$ )

e.g.  $A = \begin{pmatrix} 7 & 3 \\ 3 & -1 \end{pmatrix}$   $\lambda^2 - 6\lambda - 16 = 0 \Rightarrow \lambda = 8, -2$

Find  $v$  :  $(A - \lambda I)v = 0$  solve for each  $\lambda$  to get their  $v$ .

e.g. for  $\lambda = 8$ ,  $\begin{pmatrix} 7-8 & 3 \\ 3 & -1-8 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \Rightarrow x_1 = 3x_2 \rightarrow v = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$   
for  $\lambda = -2$ , similarly,  $v = \begin{pmatrix} 1 \\ -3 \end{pmatrix}$

Steps of PCA: 1. Get mean and get difference:  $\frac{x_i - \mu_j}{\sigma_{x_i}}$   $\rightarrow$  DataAdjust

2. Calculate covariance matrix.  $\begin{pmatrix} \text{cov}(f_1, f_1) & \text{cov}(f_1, f_2) \\ \text{cov}(f_2, f_1) & \text{cov}(f_2, f_2) \end{pmatrix}$   $\rightarrow$  for 2 features

can take unit vectors of  $v$  to get normalized form

3. Calc its eigen val & vector.  $\begin{pmatrix} \lambda_1 \\ \lambda_2 \end{pmatrix} \rightarrow \begin{pmatrix} v_1 & v_2 \\ v_1 & v_2 \end{pmatrix}$

in descending order  $\rightarrow$   $F_v = \text{Feature Vector}$

4. New dataset:  $\text{FinalDataSet} = F_v^T \times \text{DataAdjust}^T$

$(AB)^T = B^T A^T$

$\hookrightarrow$  transpose it



Ex:

| Data            |            | Data Adjust     |                 |
|-----------------|------------|-----------------|-----------------|
| x               | y          | $x_i - \bar{x}$ | $y_i - \bar{y}$ |
| 2.5             | 2.4        | 0.69            | 0.49            |
| 0.5             | 0.7        | -1.31           | -1.21           |
| 2.2             | 2.9        | 0.39            | 0.99            |
| ...             | ...        | ...             | ...             |
| mean $\bar{x}$  | $\bar{y}$  |                 |                 |
| s.d. $\sigma_x$ | $\sigma_y$ |                 |                 |

1.81

$$\text{Cov} = \begin{pmatrix} \text{cov}(x, x) & \text{cov}(x, y) \\ \text{cov}(y, x) & \text{cov}(y, y) \end{pmatrix} = \begin{pmatrix} 0.61656 & 0.61544 \\ 0.61544 & 0.71656 \end{pmatrix}$$

$$\frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{n-1} = \frac{0.69 \times 0.49 + \dots}{10-1} = 0.615444$$

Find:  $\lambda^2 - 1.333\lambda + 0.063 = 0$

$\lambda = \begin{pmatrix} 1.284 \\ 0.0491 \end{pmatrix}$ ,  $F_v = \begin{pmatrix} 0.678 & 0.735 \\ 0.735 & -0.678 \end{pmatrix}$

take top n eigenvalues & sort in descending order (here we took both)

we could've just taken the highest one too  
finalData will only have one feature

with highest  $\lambda$  used to determine the principal comp. of the data

| PC <sub>1</sub> | PC <sub>2</sub> |
|-----------------|-----------------|
| x               | y               |
| 0.828           | 0.175           |
| -1.778          | -0.143          |
| 0.992           | -0.384          |
| ...             | ...             |

$$\text{FinalData} = F_v^T \times \text{DataAdjust}^T$$

$$= \begin{pmatrix} 0.678 & 0.735 \\ 0.735 & -0.678 \end{pmatrix} \begin{pmatrix} 0.69 & -1.31 & 0.39 & \dots \\ 0.49 & -1.21 & 0.99 & \dots \end{pmatrix}$$

$$= \begin{pmatrix} 0.828 & -1.778 & 0.992 & \dots \\ 0.175 & -0.143 & -0.384 & \dots \end{pmatrix}$$



Getting original data back: (only if we took all the  $w$  in calc) otherwise we lose some info

$$\text{FinalData} = F_v^T \times \text{DataAdjust}^T$$

$$\Rightarrow F_v \times \text{FinalData} = \text{DataAdjust}^T$$

$$\Rightarrow \text{OriginalData}^T = (F_v \times \text{FinalData}) + \text{OriginalMean}$$

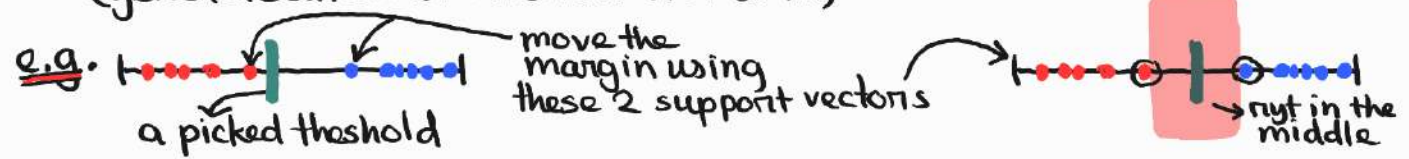
## Support Vector Machine (SVM):

Supervised learning which uses support vector for classification and regression.

data points that are closer to hyperplane and influences its pos and orient. Maximise the margin

$$y = w_1 x_1 + w_2 x_2 + w_3 x_3 + c$$

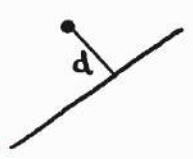
(generalisation of line for nth dim)



Hyperplane eq<sup>n</sup>,  $H: w^T \cdot x + b = 0$

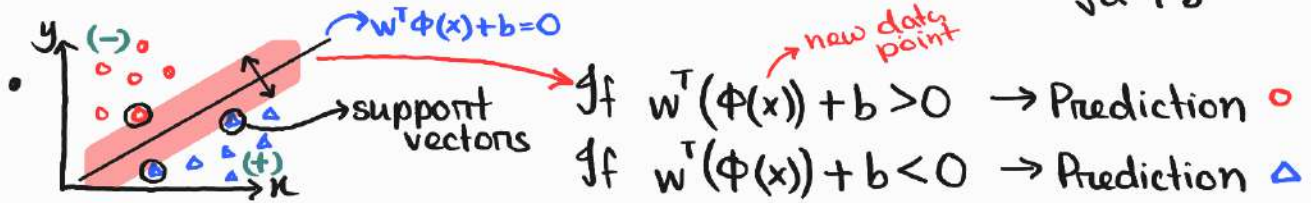
Dist from a pt to hyperplane,  $d_H(\Phi(x_0)) = \frac{|w^T \cdot \Phi(x_0) + b|}{\|w\|_2}$

$\sqrt{w_1^2 + w_2^2 + w_3^2 + \dots}$





for pt  $(x_0, y_0)$  and line  $ax+by+c=0$  :  $d = \frac{|a \cdot x_0 + b \cdot y_0 + c|}{\sqrt{a^2 + b^2}}$



Maximize the min dist,

$$w^* = \arg_w \max [\min_n d_H(\Phi(x_n))] \quad \text{Objective func of optimization}$$

$$= \arg_w \max \frac{1}{\|w\|_2} [\min_n y_n |w^T \cdot \Phi(x) + b|] \quad \text{labels: } -1, +1$$

$$= \arg_w \max \frac{1}{\|w\|_2} \quad \text{rescale the dist to get } = 1 \quad \text{all preds correct}$$

(if data is perfectly separable and all preds correct)  $\rightarrow = \min_w \frac{1}{2} \|w\|_2^2$  (if  $y_n [w^T \Phi + b] < 0 \rightarrow$  incorrect pred)  
eg.  $\frac{1}{2}$  but  $\frac{1}{2} = -ve$  if  $\geq 0 \rightarrow$  correct

Optimize with Lagrange multiplier :  $L = \frac{1}{2} \|w\|^2 - \sum_i \alpha_i (y_i (\vec{w}_i \cdot \vec{x}_i + b) - 1)$

$$\frac{\partial L}{\partial b} = 0 \Rightarrow \sum_i \alpha_i y_i = 0, \quad \alpha_i = 0$$

$$\frac{\partial L}{\partial w} = 0 \Rightarrow \sum_i \alpha_i y_i \vec{x}_i = \vec{w}, \quad \alpha_i > 0$$

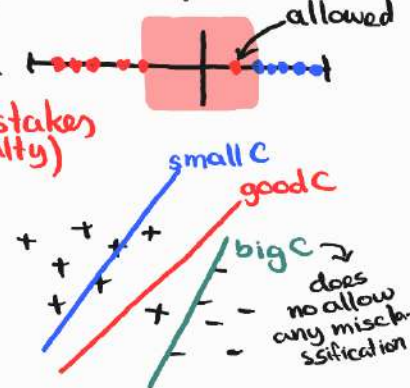
Soft margin : when we allow a few misclassification.

(not separable)

$$\min_w \frac{1}{2} \|w\|^2 + C \quad \text{no. of mistakes (penalty)}$$

$$\min_{w, b, \xi \geq 0} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

slack var  
if margin  $\geq 1$ , no penalty  
if margin  $< 1$ , pay penalty



SVM in Natural form :

SVM tries to find best  $w, b$

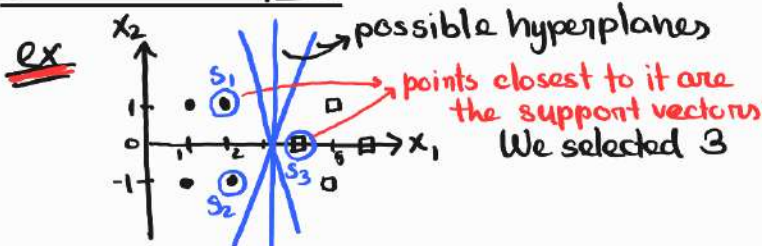
$$\arg \min_{w, b} \frac{1}{2} w \cdot w + C \sum_{i=1}^n \max \{0, 1 - y_i (w \cdot x_i + b)\} \quad \text{Regularization Parameter}$$

Margin

Empirical Loss L (how well we fit training data)  
Hinge Loss

$$\nabla(j) = \frac{\partial f(w, b)}{\partial w^{(j)}} = w^{(j)} + C \sum_{i=1}^n \frac{\partial L(x_i, y_i)}{\partial w^{(j)}} \quad w^{(j)} \leftarrow w^{(j)} - \eta \nabla(j)$$

SVM Example :



$$s_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad s_2 = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad s_3 = \begin{pmatrix} 4 \\ 0 \end{pmatrix}$$

augmented

$$\tilde{s}_1 = \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix}, \quad \tilde{s}_2 = \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix}, \quad \tilde{s}_3 = \begin{pmatrix} 4 \\ 0 \\ 1 \end{pmatrix} \quad \text{bias input}$$

Now we have to find 3 params  $\alpha_1, \alpha_2, \alpha_3$  in :

$$\alpha_1 \tilde{s}_1 \cdot \tilde{s}_1 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_1 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_1 = -1 \quad (-ve \text{ class})$$

$$\alpha_1 \tilde{s}_1 \cdot \tilde{s}_2 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_2 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_2 = -1 \quad (-ve \text{ class})$$

$$\alpha_1 \tilde{s}_1 \cdot \tilde{s}_3 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_3 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_3 = +1 \quad (+ve \text{ class})$$

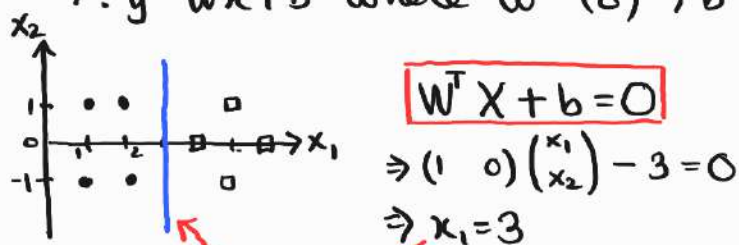
$$\begin{bmatrix} 6\alpha_1 + 4\alpha_2 + 9\alpha_3 = -1 \\ 4\alpha_1 + \dots = -1 \\ 9\alpha_1 + \dots = +1 \end{bmatrix}$$



$$\therefore \alpha_1 = \alpha_2 = -3.25, \quad \alpha_3 = 3.5$$

$$\text{Hyp plane: } \tilde{w} = \sum_i \alpha_i \tilde{x}_i = \alpha_1 \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 2 \\ -1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 4 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ -3 \end{pmatrix} \begin{matrix} \xrightarrow{w} \\ \xrightarrow{\text{offset, } b} \end{matrix}$$

$$\therefore y = wx + b \text{ where } w = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, b = -3$$



# Test case: (1, 2)

$$W^T X + b = (1 \ 0) \begin{pmatrix} 1 \\ 2 \end{pmatrix} + (-3) = -2$$

-ve. So, this point is on the left of the hyperplane.  
 $\therefore$  Belongs to • class

ex Non-linear SVM:

first it'll transform these points.

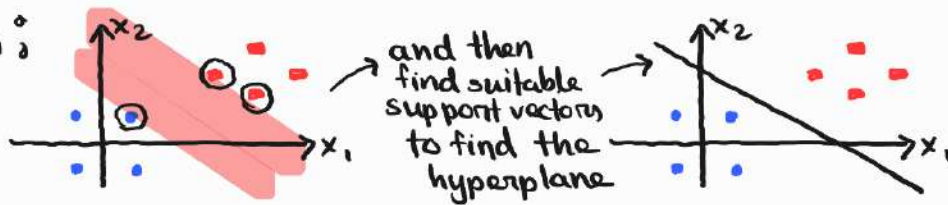
Transformation func:

(blue pt won't change as it's inside the circle  $r=2$ )

$$\Phi \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{cases} \begin{pmatrix} 6 - x_1 + (x_1 - x_2)^2 \\ 6 - x_2 + (x_1 - x_2)^2 \end{pmatrix}, & \text{if } \sqrt{x_1^2 + x_2^2} \geq 2 \\ \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, & \text{otherwise} \end{cases}$$

After transformation:

$$\text{e.g. } \Phi \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \dots = \begin{pmatrix} 8 \\ 10 \end{pmatrix}$$



• SVM Kernel:

ex.

$\rightarrow$  Can't separate with one hyperplane. Kernel is math funcs that transforms the pts to help separate.

Kernels generalise the notion of 'inner product similarity'. Not only vectors, kernels can be defined over strings, trees, etc.

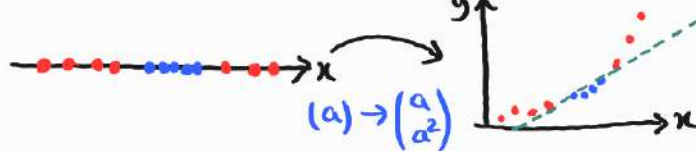
SVM with Kernel Tricks:

• Polynomial kernel,  $(a \times b + r)^d$

can use other vals too

$$\text{Taking } r = \frac{1}{2}, d = 2, \quad (a \times b + \frac{1}{2})^2 = ab + a^2 b^2 + \frac{1}{4} = \begin{pmatrix} a \\ a^2 \\ 1/2 \end{pmatrix} \cdot \begin{pmatrix} b \\ b^2 \\ 1/2 \end{pmatrix}$$

With this:



1D  $\rightarrow$  2D

• Using transform



• Non-linear



Use Polar Coords

• Other kernel funcs:  $K(x, y) = (x \cdot y + 1)^p \rightarrow$  Polynomial

gaussians  $\leftarrow K(x, y) = \exp \left\{ -\frac{\|x - y\|^2}{2\sigma^2} \right\} \rightarrow$  radial basis func

neural net activation func  $\leftarrow K(x, y) = \tanh(\kappa x \cdot y - \delta) \rightarrow$  sigmoid